Hardware-in-the-Loop Actuator Mirroring for Power Line UAV Grippers: A micro-ROS Test Bench

Letícia R. Pinto
CPGEI – UTFPR
Future Grid – LACTEC
Curitiba, Brazil
leticia.pinto@lactec.com.br

Carlos Eduardo Magrin
Future Grid – LACTEC
Curitiba, Brazil
carlos.magrin@lactec.com.br

Lucas N. Caselatto
Energy Solutions – LACTEC
Curitiba, Paraná, Brazil
lucas.caselatto@lactec.com.br

André Felipe D. Ferreira
Future Grid – LACTEC
Curitiba, Brazil
andre.ferreira@lactec.com.br

Marco A. Wehrmeister
CPGEI – UTFPR
Curitiba, Brazil
wehrmeister@utfpr.edu.br

***Abstract*—Validating autonomous manipulation tasks with uncrewed aerial vehicles (UAVs) involves significant risks of mechanical damage due to the flight instability these tasks can cause. In this scenario, simulation-based validation is a safer approach; however, failures such as communication instability and mechanical faults may not be reproducible in the virtual environment. This paper, therefore, presents an HIL test bench with actuator mirroring that integrates a virtual flight simulator (ROS 2/Gazebo) with a physical bench (ESP32/micro-ROS) in real time. The system mirrors the actuator commands of the simulated vehicle to a physical gripper mechanism on a desk. Only the gripper is physical; the flight dynamics, contact detection, and docking success remain entirely virtual. This coupling is unidirectional: the physical state is logged and compared with the virtual reference, but does not feed back into the simulated dynamics or control decisions. Experimental results with wired USB serial and wireless Wi-Fi/UDP transport show how actuator and transport-layer bottlenecks, transmission delays, breakaway static friction, and protocol-layer latency plateaus are quantified, exposing effects that may be invisible in pure simulation.**

***Index Terms*—Hardware-in-the-loop, HIL, actuator mirroring, micro-ROS, ROS 2, aerial manipulation, power line inspection, gripping mechanism.**

I. Introduction

In this work, we consider a multirotor aerial manipulator based on the DJI Matrice 400 [1], whose task is to autonomously dock onto an overhead power line conductor (Fig. 1). In the simulation presented here, the target is an energized conductor of the transmission tower; the final application of the project, however, targets the optical ground wire (OPGW) for docking and locomotion. This operation requires the vehicle to detect contact with the conductor and actuate a gripper mechanism within a narrow time window, during which the gripper must close before the vehicle loses lift or drifts laterally due to wind or other external disturbances.



Fig. 1: Multirotor aerial manipulator docked onto an overhead power line in the Gazebo simulation environment.

Validating this type of maneuver in flight is risky, as failure can damage the vehicle, the gripper, or the infrastructure itself. For this reason, Hardware-in-the-Loop (HIL) approaches have been used to validate aerial systems under controlled conditions by integrating physical hardware with a simulation environment [2], [3]. The literature, however, typically models the link between the controller and the external hardware as a channel of negligible latency. This assumption is incompatible with docking operations, in which the interval between contact detection and gripper closure is a design variable subject to temporal constraints.

The choice of a simulator in a HIL framework requires reconciling the model's physical fidelity with computational cost. Photorealism-oriented platforms, such as NVIDIA Isaac Sim, offer high visual and physical fidelity but demand substantial computational resources [4]. This work adopts Gazebo, integrated with the PX4 flight controller, for its native ROS 2 integration and computational cost that is compatible with the hardware employed, while maintaining sufficient physical

This work has been fully funded by the project *Sistema Híbrido para Inspeção Autônoma Preditiva e Detecção de Anomalias em LTs*, supported by Future Grid – Centro de Competência Embrapii Lactec em Smart Grid e Eletromobilidade, with financial resources from the PPI HardwareBR of the MCTI grant number 054/2023, signed with EMBRAPII. Dr. Wehrmeister is grateful for the financial support of the Brazilian R&D agencies Coordenação de *Aperfeiçoamento de Pessoal de Nível Superior* (CAPES – Finance Code 001) and *Fundação Araucária* (PRD2023361000420).

realism for the control tasks considered in this work [5]. The flight and contact models employed were not validated against experimental data: Gazebo is used as a reference environment for actuator control.

Despite simulator advancements, the "sim-to-real" gap remains a persistent challenge [6]–[8]. Purely virtual environments often fail to capture unmodeled physical and communication-layer realities, including static friction, timing delays, action delivery latency, and network jitter [9], [10]. To expose this gap, we present an HIL test bench with actuator mirroring that couples a virtual flight simulator (ROS 2/Gazebo) to a physical bench (ESP32/micro-ROS) to evaluate an autonomous aerial docking mechanism. By keeping the flight dynamics in Gazebo and mirroring only the gripper's actuator commands to a physical, stepper-motor-driven bench, we isolate the actuator and transport-level effects that pure simulation omits. A dedicated comparator node dynamically evaluates the desynchronization between virtual commands and physical states. Although the literature frequently frames systems such as this as digital twins, the coupling here is unidirectional: the physical state does not feed back into the simulated dynamics, as it will be discussed in Section III.

The principal contributions of this work are as follows:

- A documented software architecture for actuator command mirroring between ROS 2/Gazebo and an ESP32 via micro-ROS, with return telemetry for comparison purposes. The coupling is unidirectional: the physical state does not feed back into the simulated dynamics.
- A practical account of the integration challenges encountered, including packet jitter over Wi-Fi, stepper motor scaling, and limit switch debouncing.
- A replicable, low-cost HIL blueprint for aerial-manipulation test benches.

The remainder of this paper is organized as follows. Section II reviews related work on aerial manipulation simulators and HIL frameworks. Section III describes the proposed case study, system architecture, and communication flow. Section IV presents experimental results and transport-layer benchmarking. Section V discusses the lessons learned, and Section VI presents conclusions and future directions.

II. Related Work

A. Simulators for aerial manipulation

Previous works have simulated UAVs performing physical interaction tasks, such as object manipulation, perching on structures, and contact-based inspection [11], [12]. Surveys such as [6], [13]–[15] categorize UAV simulators into four groups: general-purpose (Gazebo, Webots, CoppeliaSim), photorealism-oriented (AirSim, Flightmare, FlightGoggles, Isaac Sim), learning-oriented (MuJuCo, PyBullet), and those prioritizing aerodynamic accuracy (X-Plane, MATLAB UAV Toolbox).

These environments operate under the assumption of fault-free software, ideal communication, and instantaneous actuator response [14], [16]. Consequently, they do not reflect the effects of deploying control logic on a physical microcontroller, where scheduling jitter, network latency, and message reordering play significant roles [3]. This limitation is particularly significant for gripper mechanisms in aerial docking, where the interval between contact detection and actuator closure is critical to the success of the operation [17].

B. Digital twins and scope of this work

The characterization of a system as a digital twin depends on the adopted criterion. Kritzinger et al. [18] distinguish three levels of integration based on the direction and automaticity of data flow, reserving the designation *digital twin* for bidirectional, automatic couplings; systems with data flowing only from the physical to the virtual are classified as digital shadows. Tao et al. [19] propose a functional maturity scale in which the initial levels correspond to basic mirroring of a component, and classify digital twins into hierarchical levels of unit, system, and system of systems [20].

Under this criterion, the system proposed in this work does not constitute a digital twin: the physical telemetry is logged and compared against the virtual reference but does not feed back into the simulated dynamics or the control decisions, thereby characterizing it as a digital shadow. Furthermore, only the gripper subsystem is physically mirrored, placing this work at the unit level and at the initial levels of the maturity scale. We therefore adopt the designation "HIL test bench" with unidirectional mirroring throughout this article.

C. HIL frameworks for aerial systems

Standard HIL setups for UAVs validate flight controllers and obstacle avoidance by feeding physical vehicle positions into simulators through motion capture [3]. These frameworks, however, typically treat external physical tools and serial communication as instantaneous. Research on robotic grippers in ROS 2 has shown that simulated actuators lack force feedback, and their inherent opening delays alter object placement [14]. To expose communication faults, Djabin et al. [21] forced simulated telemetry through physical ESP32 modules, confirming that message reordering and scheduling jitter are real concerns.

The literature on HIL frameworks for aerial manipulation has primarily focused on flight controllers and navigation, with less attention given to actuator-level synchronization using embedded microcontrollers. The case study presented here investigates how micro-ROS on an ESP32 performs as an actuator-level interface, specifically examining how different transports (USB serial vs. Wi-Fi/UDP) affect message reordering and fault detection. This work does not claim novelty in the coupling itself, but empirically documents the observed timing effects, establishing a baseline for future research on actuator-level HIL validation.

III. Case Study Description

This section details the case study designed to evaluate the proposed HIL test bench during autonomous power line docking maneuvers. The setup integrates a virtual flight domain

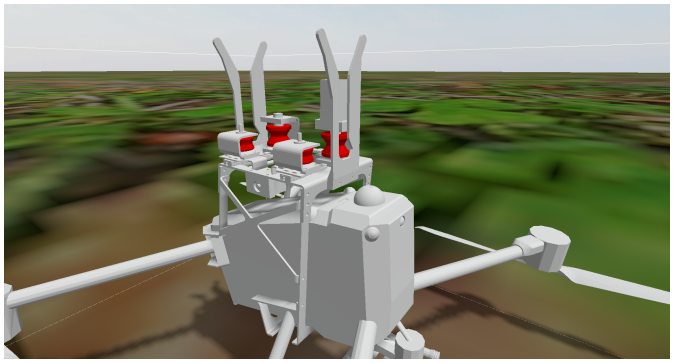


Fig. 2: Virtual Twin model of the power line docking mechanism, rendered on the upper bay of the UAV within the Gazebo simulation environment.

in Gazebo with a physical actuator bench, powered by micro-ROS on an ESP32, to analyze command-tracking fidelity and transport-layer latency under operational constraints. The following subsections present the simulation scenario and flight operational phases, the multi-domain system architecture, and the asynchronous communication pipeline linking the virtual and physical layers.

A. Simulation Scenario

The target scenario comprises a DJI Matrice 400 multirotor equipped with an upward-facing gripper mechanism designed for autonomous docking and inspection of transmission lines. Fig. 2 shows the virtual models of this mechanism, rendered on the upper bay of the UAV in the simulation environment. Fig. 3 shows the complete simulation environment, composed of three transmission towers connected by rigid conductor cables, representing a segment of the power distribution infrastructure.



Fig. 3: Gazebo simulation scenario, showing the three transmission towers and the conductor cables that form the power distribution infrastructure segment

To perform a successful docking operation, the UAV must execute a sequential control pipeline divided into three main operational phases:

- Navigate to a stable hovering position directly below the target transmission line;
- Ascend vertically until the Gazebo contact sensors register contact;

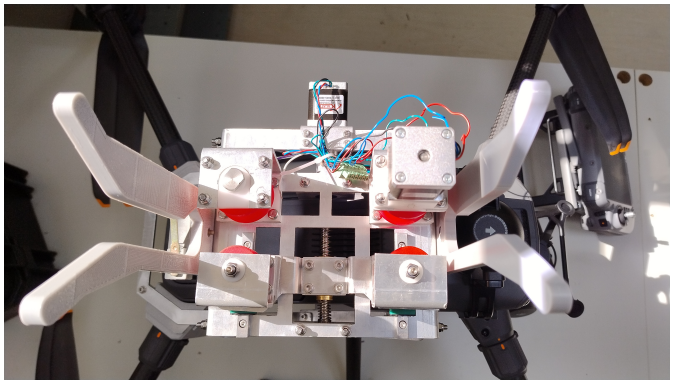


Fig. 4: Physical Twin mechanism top view detailing the dual-actuator layout, the central lead-screw mechanism for prismatic execution, and the symmetric distribution of the contact rollers.

- Trigger and fully close the gripper mechanism within a bounded time window after contact.

The physical counterpart of the mechanism is shown in Fig. 4, which details the actuator layout, the central lead screw for prismatic execution, and the symmetric distribution of the contact rollers.

The coordination layer establishes an asynchronous communication pipeline that forwards joint position commands from Gazebo to the physical ESP32 and returns actuator telemetry to the simulator. This layer performs no control correction or fault mitigation; it is strictly dedicated to telemetry collection and observation. A comparator node measures the positional error and the transport-layer latency between the virtual reference and the physical state in real time. This configuration enables the empirical quantification of actuators and transport-level effects, such as breakaway static friction and network delays, under controlled bench conditions.

B. System Architecture

The architecture consists of two domains connected by a ROS 2/micro-ROS bridge (Fig. 5) :

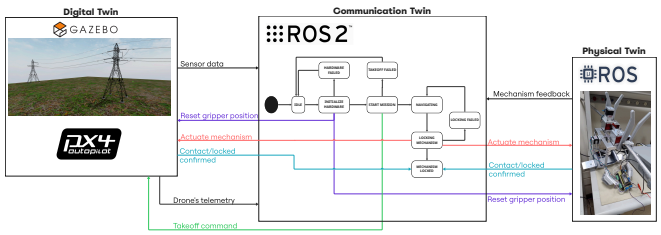


Fig. 5: Overview of the hybrid digital twin system architecture, illustrating the bidirectional communication flow between the virtual flight domain (ROS 2/Gazebo), the coordination layer, and the physical embedded layer (micro-ROS/ESP32)

- Virtual domain: This domain simulates a multirotor in Gazebo with an upward-facing prismatic joint acting as

the gripper mechanism. The model treats the coupler as an ideal, frictionless system that responds instantaneously to continuous position commands. The active ROS 2 nodes include the flight controller, the gripper trajectory generator, and an interface node that mirrors the simulated positions to the physical bench.

- Physical domain: The bench is powered by an ESP32 running a native micro-ROS client, with a NEMA 17 stepper motor drive, a lead-screw transmission, and two limit switches that bound the extremes of a 43.0 mm stroke. This domain introduces mechanical imperfections not present in the virtual model, such as backlash and lead-screw friction, which restrict and delay motion. The physical position reported in this work is inferred from the step count executed by the ESP32; there is no independent encoder, and the limit switches provide only endpoint verification. Effects such as missed steps, backlash, and lead-screw compliance are therefore not observable in the data.
- Hybrid domain: The coordination layer reconciles a fundamental difference between the domains: the continuous, closed-loop position controller of Gazebo on one side, and the discrete, open-loop step-pulse sequences of the ESP32 on the other. The layer subscribes to the continuous virtual position and maps it linearly to motor target steps (from 800 down to 0 steps, covering a 43.0 mm stroke). The comparator node analyzes the response times.

C. Communication Flow

To connect the virtual environment and the physical hardware, an asynchronous communication pipeline is established. While Gazebo operates on simulated time and the ESP32 on its internal hardware clock, the micro-ROS agent timestamps outgoing and incoming packets using the ROS 2 system time. The comparator node uses this metadata to calculate and log transport-layer latencies. The pipeline operates as follows:

- Gazebo detects contact between the vehicle's rollers and the conductor mesh, which triggers the autonomous docking sequence.
- The gripper command generator initiates the closing phase by publishing a continuous virtual joint trajectory from 0.0 to 0.043 m at a fixed rate of 50 Hz. The corresponding profile in the step domain decreases monotonically from 800 to 0 steps, at a calibrated steady-state velocity of 49.833 steps/s.
- The coordination node maps continuous coordinates to discrete target-step pulses and packages them into a ROS 2 message, which is forwarded to the ESP32 via micro-ROS (USB serial or Wi-Fi/UDP).
- Upon receiving the target steps, the ESP32 drives the physical stepper motor.
- The ESP32 packages the executed step count and the limit switch states into a feedback packet and streams it back to the ROS 2 coordination layer.
- The comparator node receives the feedback, aligns it with the virtual reference trajectory using the ROS 2 arrival time, and calculates the absolute tracking error and communication latency for offline analysis.

IV. Experimental Results and Evaluation

This section presents the empirical evaluation of the HIL test bench. The platform was executed continuously at a fixed ROS 2 control loop sampling frequency of 20 Hz, giving a discrete sampling period of $T_s = 50$ ms. To isolate transport-layer effects and study their impact on spatial-temporal tracking, the entire pipeline was benchmarked under two transport modalities:

- Wired USB serial: Interfaced via a dedicated micro-ROS serial agent loop running at a deterministic rate of 115200 bps.
- Wireless Wi-Fi/UDP: Dispatched via a native wireless UDP socket over a local IEEE 802.11b/g/n (Wi-Fi) network operating at 2.4 GHz.

A. Spatial-Temporal Synchronization Profiles

To evaluate the tracking capability of the test bench, the transient positions of the physical joint are compared against the virtual Gazebo reference in Fig. 6. The left column shows the wired USB serial connection; the right column shows the wireless Wi-Fi/UDP link.

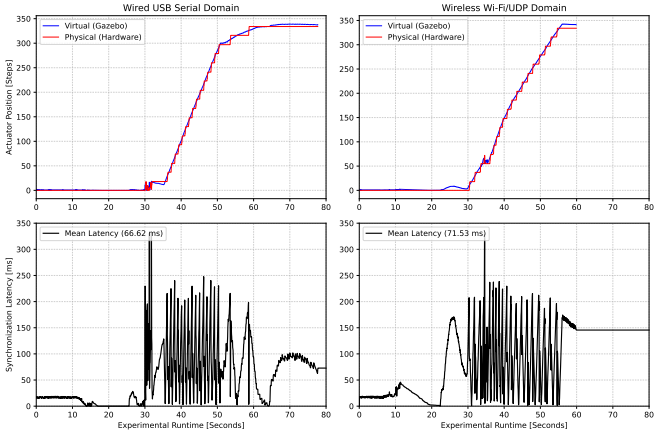


Fig. 6: HIL synchronization analysis under wired USB serial and wireless Wi-Fi/UDP transport. The top row compares the step-position tracking between the virtual model (Gazebo) and the physical actuator. The bottom row shows the corresponding mean transport latency over the experimental timeline.

The physical position tracks the virtual reference within the measured error range. During active translation ($t \approx 35$ s to 55 s), the physical position trajectory (red curve) closely follows the Gazebo joint reference (blue curve). The staircase patterns in the physical response reflect the discrete step resolution of the NEMA stepper motor and the ESP32 sampling rate, in contrast with the continuous joint space simulated in Gazebo.

A transient maximum peak error of 16.63 steps (0.894 mm) occurs shortly after motion onset, at $t = 31.39$ s, in both runs. This peak corresponds to the breakaway static friction of the lead-screw assembly, an effect not represented in the virtual model. The comparator node registers this breakaway friction as a temporal offset at the onset of motion, indicating an actuator-level effect that the virtual model does not represent.

The communication layer shows distinct behaviors between the two transports. The wired USB serial link reaches a mean latency of 66.62 ms, with a stable baseline near 20 ms during the stationary phase (0 s to 25 s). During active motion, it exhibits queuing and transient spikes approaching 300 ms, peaking at 333.22 ms, due to high-frequency serialization. Once motion concludes ($t > 60$ s), the wired link recovers to approximately 80 ms.

The wireless Wi-Fi/UDP link exhibits a higher mean latency of 71.53 ms and a higher tracking error (Table I). The transient peak is slightly lower than that of the serial link (330.76 ms versus 333.22 ms). The most marked difference between the two transports appears in the final steady-state hold phase ($t > 60$ s). While the wired link recovers to a lower delay, the wireless link remains at an elevated plateau of approximately 150 ms. This persistent offset is consistent with packet buffering effects or unrecovered delays in the UDP stack under continuous transmission.

B. Comparative Transport Benchmarking

Given a discrete sampling period of $T_s = 0.05$ s over a synchronized 80 s window, the error signal is defined as the difference between the virtual joint reference command from Gazebo and the physical step position tracked by the ESP32 at each sample index. The physical position is inferred from the executed step count, not from an independent encoder; effects such as missed steps, backlash, and lead-screw compliance are therefore not captured by this metric. The reported RMSE values quantify command-to-execution tracking, not the true mechanical displacement error, and are not a measure of the complete UAV's geometric docking accuracy.

The total trajectory deviation was quantified using the Root Mean Square Error (RMSE). The step-domain metrics were converted to metric units using the calibration factor derived from the lead-screw pitch, where 80 steps correspond to 4.3 mm of linear travel (approximately 0.05375 mm per step). The spatial tracking delays were projected back into the temporal domain using the calibrated steady-state actuator velocity of 49.833 steps/s, yielding the Mean Steady-State Latency and Peak Transient Latency values.

The performance indicators across both transports are summarized in Table I.

The evaluation shows that the wireless Wi-Fi link preserves sub-millimeter command-tracking precision, with a global RMSE increase of only 0.0829 mm (from 0.2117 mm to 0.2946 mm). This small variance indicates that the wireless hardware can track the commanded actuator trajectory within the measurement resolution of the step-count feedback. Temporally, the wireless link introduces an average latency overhead of 4.91 ms relative to the serial baseline. Both transports meet the baseline geometric and temporal requirements for actuator-level validation.

TABLE I: Actuator Tracking Performance Across Transport Layers

Performance Indicator	**Wired USB Serial**	**Wireless Wi-Fi/UDP**
RMSE [steps]	3.9392	5.4801
RMSE [mm]	0.2117	0.2946
Mean Steady-State Latency [ms]	66.62	71.53
Peak Transient Latency [ms]	333.22	330.76

V. Discussion and Lessons Learned

The primary value of projecting virtual tracking streams onto physical hardware lies in exposing actuator- and transport-level effects that purely virtual simulators omit, with implications for system reliability.

A. Analysis of the Communication Layer

Communication noise is particularly evident during the steady-state hold phase ($t \geq 40.42$ s), where packet jitter and buffering delays produce continuous latency fluctuations. Unlike the wired serial connection, which returns to a low, stable latency baseline once movement ceases, the wireless Wi-Fi/UDP link remains at an elevated latency of approximately 150 ms. This persistent offset is consistent with buffering effects in the UDP stack after high-rate transmission bursts.

B. Embedded Integration Challenges

Integrating Gazebo with the ESP32 via micro-ROS presented three challenges arising from the device's resource limitations:

- Connection robustness: Connection dropouts and packet loss over wireless UDP demanded additional handling. In the absence of the automatic recovery mechanisms typical of wired protocols, such situations could require manual device resets or lead to interruption of the main loop. This behavior motivated the implementation of recovery mechanisms in the final version.
- Continuous-to-discrete mapping: Converting continuous joint trajectories from Gazebo (50 Hz) into discrete step commands for the physical motor proved non-trivial. Abrupt velocity changes could exceed the motor's acceleration limits, requiring empirical scaling adjustments to prevent mechanical stalling.

These results indicate that the idealized physics of the virtual model omits effects relevant to control design. The virtual joint in Gazebo does not exhibit the same startup delay observed in the physical joint, which registers a 330 ms latency peak at motion onset ($t \approx 31.39$ s). This delay is consistent with the breakaway static friction of the lead-screw assembly, which requires a buildup of motor torque before rotation begins.

C. Implications for Flight Control

In an actual flight mission, this latency defines the lower bound of the flight controller's temporal safety window. If the controller assumes instantaneous gripper actuation, it may initiate tensioning or takeoff maneuvers before the gripper is structurally locked onto the line. Verifying the actual consequences would require the coupled flight experiments listed as future work.

VI. Conclusion and Future Work

This work presented a HIL test bench that couples a virtual flight simulation with a physical, low-level control architecture, keeping flight dynamics and contact detection in the virtual environment while validating actuator tracking on physical hardware. The system maintained command tracking within the reported bounds, with command-to-execution RMSEs in the sub-millimeter range. The main contribution of the bench is to expose actuator- and transport-level effects that are not represented in purely virtual simulations.

Future work will address four directions:

- Encoder integration: Adding an independent encoder to the bench to measure true mechanical displacement and quantify missed steps, backlash, and lead-screw compliance.
- Network degradation stress-testing: Characterizing the failure boundaries of the wireless micro-ROS link under packet loss, RSSI attenuation, and network congestion, to define safety-stop thresholds.
- Force-feedback integration: Implementing torque or current sensing on the bench to stream hardware resistance back into Gazebo, enabling closed-loop experiments.
- Expansion toward a digital twin: The current bench operates as a digital shadow, with unidirectional flow. The next step is to close the loop, so that the physical state alters the simulated dynamics and the control decisions. The encoder and force-feedback integrations are prerequisites for this expansion.

Acknowledgment

This work has been fully funded by the project *Sistema Híbrido para Inspeção Autônoma Preditiva e Detecção de Anomalias em LTs*, supported by Future Grid – Centro de Competência Embrapii Lactec em Smart Grid e Eletromobilidade, with financial resources from the PPI HardwareBR of the MCTI grant number 054/2023, signed with EMBRAPII.

References

[1] "DJI Matrice 400 - Engineered for Excellence, Designed for Versatility - DJI Enterprise," DJI. Accessed: Sep. 21, 2026. [Online]. Available: https://enterprise.dji.com/photo

[2] A. Oliveira et al., "Simulation Environment for UAV Offshore Wind-Turbine Inspection," in OCEANS 2023 - Limerick, Limerick, Ireland: IEEE, Jun. 2023, pp. 1–6. doi: 10.1109/OCEANSLimerick52467.2023.10244328.

[3] D. Dose, M. Tappe, M. Alpen, and J. Horn, "Hardware in the Loop Simulation of Aircraft Inspection by an Unmanned Aerial System," in 2021 7th International Conference on Automation, Robotics and Applications (ICARA), Prague, Czech Republic: IEEE, Feb. 2021, pp. 147–151. doi: 10.1109/ICARA51699.2021.9376532.

[4] Á. Soriano et al., "Benchmarking Full-Stack ROS 2 Simulation Platforms for Mobile Robots," IEEE Robot. Autom. Lett., vol. 11, no. 5, pp. 6242–6247, May 2026, doi: 10.1109/LRA.2026.3678463.

[5] J. Dos Santos et al., "Benchmarking Digital Twins for Tower Cranes: Isaac Sim vs. Gazebo," in IECON 2025 – 51st Annual Conference of the IEEE Industrial Electronics Society, Madrid, Spain: IEEE, Oct. 2025, pp. 1–8. doi: 10.1109/IECON58223.2025.11221823.

[6] C. A. Dimmig et al., "Survey of Simulators for Aerial Robots: An Overview and In-Depth Systematic Comparisons," 2023, doi: 10.48550/ARXIV.2311.02296.

[7] M. Mittal et al., "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments," IEEE Robot. Autom. Lett., vol. 8, no. 6, pp. 3740–3747, Jun. 2023, doi: 10.1109/LRA.2023.3270034.

[8] D. P. Leins, R. Haschke, and H. Ritter, "MuJoCo ROS: Integrating ROS with the MuJoCo Engine for Accurate and Scalable Robotic Simulation," in 2025 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR), Palermo, Italy: IEEE, Apr. 2025, pp. 1–6. doi: 10.1109/SIMPAR62925.2025.10979045.

[9] S. Akki and T. Chen, "Benchmarking Model Predictive Control and Reinforcement Learning Based Control for Legged Robot Locomotion in MuJoCo Simulation," 2025, arXiv. doi: 10.48550/ARXIV.2501.16590.

[10] A. Handa et al., "DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality," 2022, arXiv. doi: 10.48550/ARXIV.2210.13702.

[11] A. Scalvini, A. Suarez, S. R. Nekoo, and A. Ollero, "Flapping-Wing Aerial Manipulation Robot with Perching-Launching Capabilities: Integrated Modeling and Control," in Robot 2023: Sixth Iberian Robotics Conference, vol. 978, L. Marques, C. Santos, J. L. Lima, D. Tardioli, and M. Ferre, Eds., in Lecture Notes in Networks and Systems, vol. 978. , Cham: Springer Nature Switzerland, 2024, pp. 98–109. doi: 10.1007/978-3-031-59167-9_9.

[12] O. B. Schofield, K. H. Lorenzen, and E. Ebeid, "Cloud to Cable: A Drone Framework for Autonomous Power Line Inspection," in 2020 23rd Euromicro Conference on Digital System Design (DSD), Kranj, Slovenia: IEEE, Aug. 2020, pp. 503–509. doi: 10.1109/DSD51259.2020.00085.

[13] A. I. Hentati, L. Krichen, M. Fourati, and L. C. Fourati, "Simulation Tools, Environments and Frameworks for UAV Systems Performance Analysis," in 2018 14th International Wireless Communications & Mobile Computing Conference (IWCMC), Limassol: IEEE, Jun. 2018, pp. 1495–1500. doi: 10.1109/IWCMC.2018.8450505.

[14] F. P. Audonnet, A. Hamilton, and G. Aragon-Camarasa, "A Systematic Comparison of Simulation Software for Robotic Arm Manipulation using ROS2," 2022, arXiv. doi: 10.48550/ARXIV.2204.06433.

[15] E. Berrocal, B. Sierra, and H. Herrero, "Evaluating PyBullet and Isaac Sim in the Scope of Robotics and Reinforcement Learning," in 2024 7th Iberian Robotics Conference (ROBOT), Madrid, Spain: IEEE, Nov. 2024, pp. 1–6. doi: 10.1109/ROBOT61475.2024.10797383.

[16] Y. Yang, W. Meng, and S. Zhu, "A Digital Twin Simulation Platform for Multi-rotor UAV," in 2020 7th International Conference on Information, Cybernetics, and Computational Social Systems (ICCSS), Guangzhou, China: IEEE, Nov. 2020, pp. 591–596. doi: 10.1109/ICCSS52145.2020.9336872.

[17] S. Cocuzza, E. Rossetto, and A. Doria, "Dynamic interaction between robot and UAV in aerial manipulation," in 2020 19th International Conference on Mechatronics - Mechatronika (ME), Prague, Czech Republic: IEEE, Dec. 2020, pp. 1–6. doi: 10.1109/ME49197.2020.9286694.

[18] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital Twin in manufacturing: A categorical literature review and classification," IFAC-PapersOnLine, vol. 51, no. 11, pp. 1016–1022, 2018, doi: 10.1016/j.ifacol.2018.08.474.

[19] J.-F. Yao, Y. Yang, X.-C. Wang, and X.-P. Zhang, "Systematic review of digital twin technology and applications," Vis. Comput. Ind. Biomed. Art, vol. 6, no. 1, p. 10, May 2023, doi: 10.1186/s42492-023-00137-4.

[20] F. Tao, B. Xiao, Q. Qi, J. Cheng, and P. Ji, "Digital twin modeling," Journal of Manufacturing Systems, vol. 64, pp. 372–389, Jul. 2022, doi: 10.1016/j.jmsy.2022.06.015.

[21] J. Djabin, M. Kurenda, J. Kochan, K. Wojtowicz, P. Wojciechowski, and A. Marut, "Scalable Hardware in the Loop (HIL) System for Real-Time Swarm Drone Control Simulation," in 2024 IEEE International Workshop on Technologies for Defense and Security (TechDefense), Naples, Italy: IEEE, Nov. 2024, pp. 318–322. doi: 10.1109/TechDefense63521.2024.10863036.